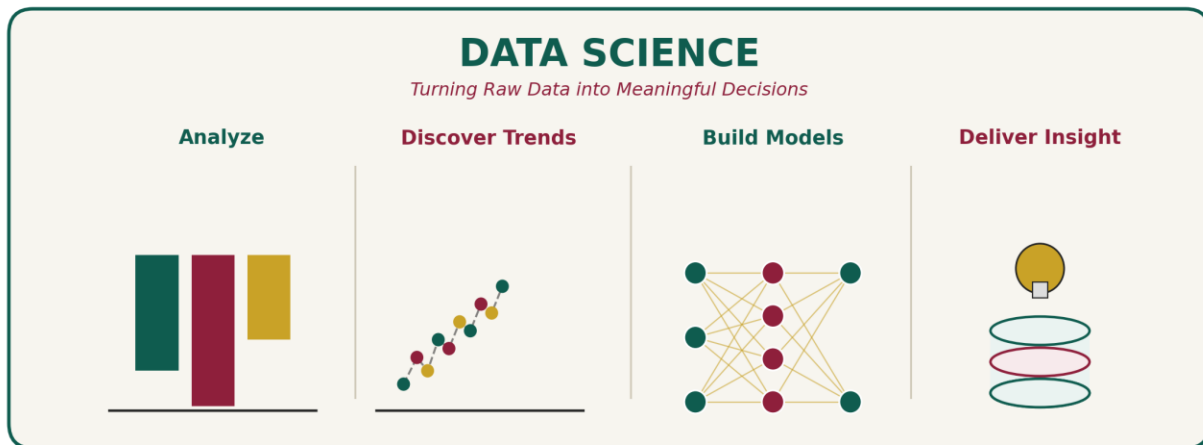


Part A: Fundamentals

Data Analysis | Data Science Workflow | ML Problem Framing | Tensors



Q1. What is Data Analysis?

Data Analysis is the systematic process of inspecting, cleaning, transforming, and modelling data with the goal of discovering useful information, drawing conclusions, and supporting decision-making. It involves applying statistical and logical techniques to raw data in order to identify patterns, relationships, and trends that are not immediately visible.

In simple words, data analysis converts raw, unorganised data into meaningful insights that help individuals and organisations make informed, evidence-based decisions rather than relying on guesswork.

Key Purposes of Data Analysis

- **Understanding patterns:** identifying trends, correlations, and anomalies within data.
- **Decision support:** providing a factual, evidence-based foundation for business and research decisions.
- **Problem solving:** diagnosing the root cause of an issue using historical or real-time data.
- **Forecasting:** predicting future outcomes using past data trends.

Q2. Planning a Data Science Project

A Data Science project is planned and executed in a structured, cyclical manner known as the Data Science Lifecycle. Each stage builds upon the previous one, and the process is often iterative — insights from later stages may require revisiting earlier ones.

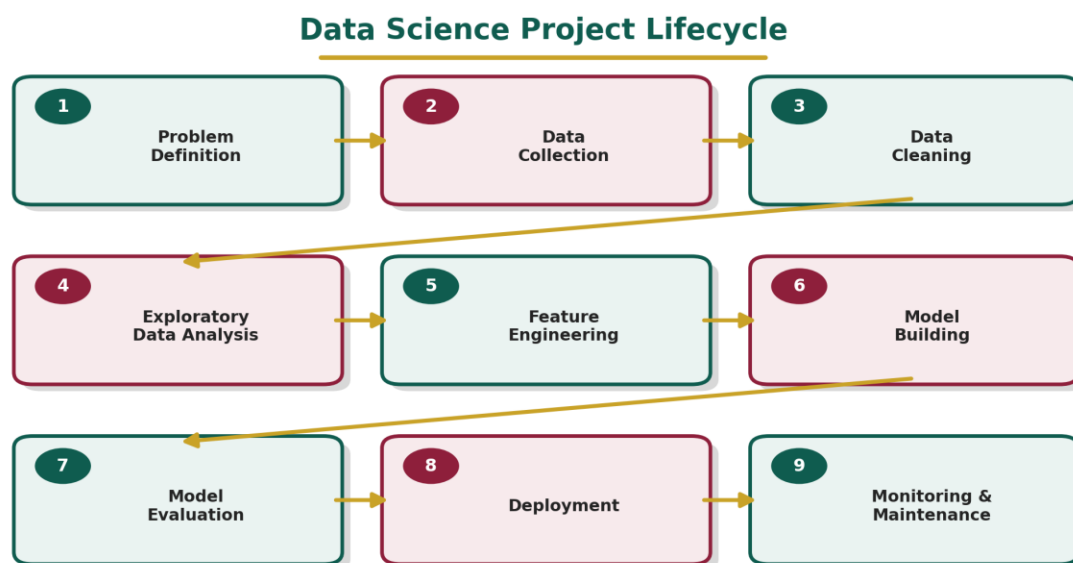


Fig 1: Data Science Project Lifecycle

Steps to Plan a Data Science Project

- 1. Problem Definition** — clearly understand the business problem, define objectives, success metrics, and the scope of the project.
- 2. Data Collection** — gather relevant data from databases, APIs, web scraping, surveys, or third-party sources.
- 3. Data Cleaning** — handle missing values, duplicates, outliers, and inconsistent formats to prepare a reliable dataset.
- 4. Exploratory Data Analysis (EDA)** — use statistics and visualisations to understand data distribution, relationships, and patterns.
- 5. Feature Engineering** — create, transform, or select the most relevant variables to improve model performance.
- 6. Model Building** — select and train suitable machine learning algorithms on the prepared data.
- 7. Model Evaluation** — assess performance using metrics such as accuracy, precision, recall, F1-score, or RMSE.

8. **Deployment** — integrate the trained model into a production environment or application for real-world use.
9. **Monitoring & Maintenance** — continuously track model performance and retrain when accuracy degrades over time.

Q3. Machine Learning Problem Statement

Problem: Predict whether a customer will churn (leave the company) based on their purchase behaviour.

Problem Type	Supervised Learning — Binary Classification
Objective	Predict the probability that a customer will stop doing business with the company (churn) using historical purchase and engagement data.
Input (Features)	Purchase frequency, recency of last purchase, average order value, total spend, number of support tickets raised, customer tenure, product category preferences, etc.
Output (Target)	A binary label — Churn = 1 (Yes, customer is likely to leave) or Churn = 0 (No, customer is likely to stay).
Suitable Algorithms	Logistic Regression, Decision Tree, Random Forest, Gradient Boosting (XGBoost/LightGBM), or Neural Networks.
Evaluation Metrics	Accuracy, Precision, Recall, F1-score, and ROC-AUC (since churn datasets are often imbalanced).
Business Value	Enables the company to proactively identify at-risk customers and take retention actions such as offers, discounts, or personalised engagement.

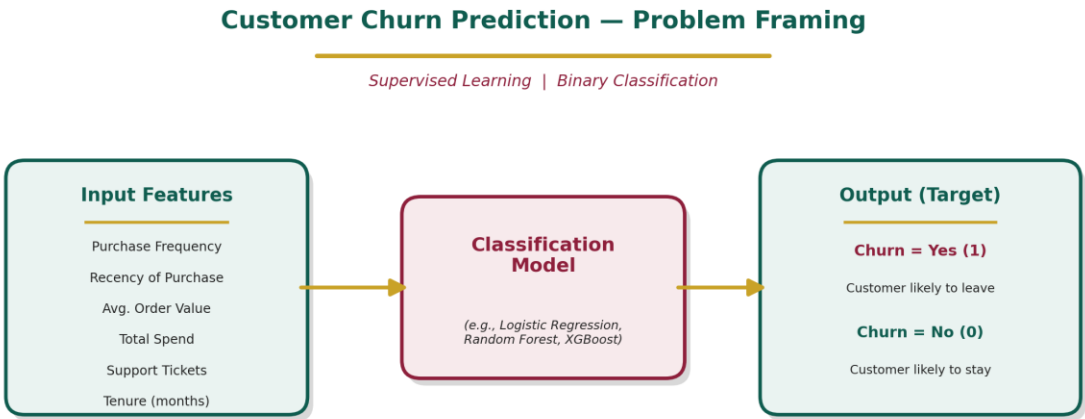


Fig 2: Churn Prediction — Problem Framing

Formal Problem Statement

“Given a customer's historical purchase and engagement data, build a supervised binary classification model that predicts whether the customer will churn (1) or not churn (0), enabling the business to take timely retention measures.”

Q4. What are Tensors?

A Tensor is a generalised, multi-dimensional container of numerical data. It is the fundamental data structure used in Machine Learning and Deep Learning frameworks (such as NumPy, TensorFlow, and PyTorch) to store and manipulate data of any dimensionality.

Tensors are characterised by their rank (also called order or number of dimensions/axes) and shape (the size along each dimension). A tensor is simply a generalisation of scalars, vectors, and matrices to higher dimensions.

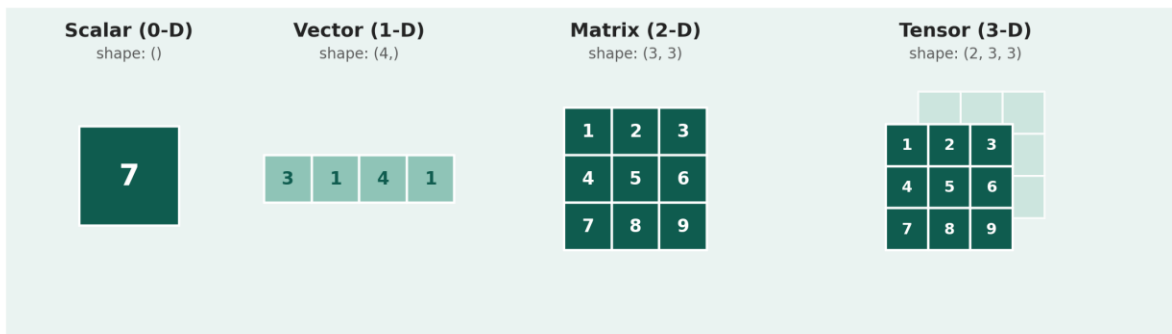


Fig 3: Tensor Hierarchy — Scalar → Vector → Matrix → Tensor

Scalar (Rank-0)	A single number with no axes. Example: 5
Vector (Rank-1)	A 1-D array of numbers. Example: [2, 4, 6]
Matrix (Rank-2)	A 2-D array (rows × columns). Example: a 2×2 table of numbers.
Tensor (Rank-3+)	A generalisation with 3 or more axes. Example: a stack of matrices, such as a colour image (height × width × channels).

In-Depth Explanation with NumPy Examples

NumPy represents tensors using the ndarray (n-dimensional array) object. Every ndarray has two key properties: `.ndim` (number of axes / rank) and `.shape` (size along each axis).

1. Scalar & Vector

```
import numpy as np

scalar = np.array(5)
vector = np.array([2, 4, 6])

print(scalar.ndim, scalar.shape)  # 0  ()
```

```
print(vector.ndim, vector.shape) # 1 (3,)
```

2. Matrix & Higher-Dimensional Tensor

```
matrix = np.array([[1, 2], [3, 4]])
tensor_3d = np.array([[[1,2],[3,4]], [[5,6],[7,8]]])

print(matrix.ndim, matrix.shape) # 2 (2, 2)
print(tensor_3d.ndim, tensor_3d.shape) # 3 (2, 2, 2)

# Real image example: (height, width, channels)
image_tensor = np.random.rand(224, 224, 3) # shape (224,224,3)
```

Why Tensors Matter in Machine Learning

- **Uniform data representation:** images, text, audio, and tabular data can all be represented as tensors of different ranks.
- **Efficient computation:** tensor operations are highly optimised for parallel processing on CPUs/GPUs.
- **Foundation of Deep Learning:** neural network inputs, weights, and outputs are all stored and processed as tensors.